Lab 4: VibeCoding

Name: Yellapantula Venkata Prateek
SRN: PES2UG24CS617
SECTION: 5J

Github link: https://github.com/Prateek-YV/SE-Lab-Submissions.git

Card.py

```python
import pygame


class Card:
    _fonts = {}

    @classmethod
    def _font(cls, size):
        if size not in cls._fonts:
            cls._fonts[size] = pygame.font.SysFont(None, size)
        return cls._fonts[size]

    def __init__(self, rank_str, suit_str, numeric_rank):
        self.rank_str = rank_str
        self.suit_str = suit_str
        self.numeric_rank = numeric_rank

        self.symbol = {"Hearts": "♥", "Diamonds": "♦", "Clubs": "♣", "Spades": "♠"}.get(suit_str, "")
        self.is_red = suit_str in ("Hearts", "Diamonds")
        self.color = (220, 40, 40) if self.is_red else (30, 30, 30)

    def render(self, surface, x, y, width=130, height=180):
        card_rect = pygame.Rect(x, y, width, height)
        pygame.draw.rect(surface, (255, 255, 255), card_rect, border_radius=10)
        pygame.draw.rect(surface, (50, 50, 50), card_rect, width=3, border_radius=10)

        font_rank = self._font(36)
        font_symbol = self._font(64)

        rank_surf = font_rank.render(self.rank_str, True, self.color)
        surface.blit(rank_surf, (x + 10, y + 8))

        self._draw_suit(surface, x + width // 2, y + height // 2, 26)

        surface.blit(rank_surf, (x + width - rank_surf.get_width() - 10, y + height - rank_surf.get_height() - 8))

    def _draw_suit(self, surface, cx, cy, s):
        """Draws the suit with shapes so it never depends on font glyph support."""
        c = self.color
        if self.suit_str == "Hearts":
            r = s // 2
            pygame.draw.circle(surface, c, (cx - r, cy - r // 2), r)
            pygame.draw.circle(surface, c, (cx + r, cy - r // 2), r)
            pygame.draw.polygon(surface, c, [(cx - s + 1, cy - r // 4), (cx + s - 1, cy - r // 4), (cx, cy + s)])
        elif self.suit_str == "Diamonds":
            pygame.draw.polygon(surface, c, [(cx, cy - s), (cx + int(s * 0.7), cy), (cx, cy + s), (cx - int(s * 0.7), cy)])
        elif self.suit_str == "Clubs":
            r = int(s * 0.42)
            pygame.draw.circle(surface, c, (cx, cy - r), r)
            pygame.draw.circle(surface, c, (cx - r, cy + r // 2), r)
            pygame.draw.circle(surface, c, (cx + r, cy + r // 2), r)
            pygame.draw.polygon(surface, c, [(cx - 4, cy + r), (cx + 4, cy + r), (cx + 7, cy + s), (cx - 7, cy + s)])
        else:  # Spades
            r = s // 2
            pygame.draw.circle(surface, c, (cx - r, cy + r // 3), r)
            pygame.draw.circle(surface, c, (cx + r, cy + r // 3), r)
            pygame.draw.polygon(surface, c, [(cx - s + 1, cy + r // 6), (cx + s - 1, cy + r // 6), (cx, cy - s)])
            pygame.draw.polygon(surface, c, [(cx - 4, cy + r), (cx + 4, cy + r), (cx + 7, cy + s), (cx - 7, cy + s)])

    @classmethod
    def render_back(cls, surface, x, y, width=130, height=180):
        """Draws a face-down card (used for the hidden next card)."""
        card_rect = pygame.Rect(x, y, width, height)
        pygame.draw.rect(surface, (35, 60, 140), card_rect, border_radius=10)
        pygame.draw.rect(surface, (240, 240, 240), card_rect, width=3, border_radius=10)
        inner = card_rect.inflate(-24, -24)
        pygame.draw.rect(surface, (70, 100, 190), inner, width=2, border_radius=6)
        q = cls._font(64).render("?", True, (240, 240, 240))
        surface.blit(q, (card_rect.centerx - q.get_width() // 2, card_rect.centery - q.get_height() // 2))
```

Deck.py

```python
import random
from game.card import Card


class Deck:

    SUITS = ["Hearts", "Diamonds", "Clubs", "Spades"]
    RANKS = [
        ("2", 2), ("3", 3), ("4", 4), ("5", 5), ("6", 6),
        ("7", 7), ("8", 8), ("9", 9), ("10", 10),
        ("J", 11), ("Q", 12), ("K", 13), ("A", 14)
    ]

    def __init__(self):
        self.cards = []
        self.reset()

    def reset(self):
        self.cards = [Card(r[0], s, r[1]) for s in self.SUITS for r in self.RANKS]
        random.shuffle(self.cards)

    def draw(self):
        if not self.cards:
            self.reset()
        return self.cards.pop()

    @property
    def remaining(self):
        return len(self.cards)
```

Gameengine.py

```python
import pygame
from game.deck import Deck


class GameEngine:
    # Game states
    WAITING = "WAITING"      # player is choosing HIGHER / LOWER
    REVEALING = "REVEALING"  # new card is being flipped / shown, input locked

    FLIP_MS = 350            # duration of the flip animation
    REVEAL_MS = 1300         # total time the result stays on screen (includes flip)
    MAX_MULTIPLIER = 5       # streak multiplier cap

    CARD_W, CARD_H = 130, 180

    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.deck = Deck()

        self.current_card = self.deck.draw()
        self.next_card = None
        self.score = 0
        self.streak = 0
        self.multiplier = 1
        self.state = self.WAITING
        self.reveal_start = 0
        self.status_msg = "Will the next card be HIGHER or LOWER?"
        self.status_color = (220, 220, 220)
        self.pending_msg = self.status_msg
        self.pending_color = self.status_color

        btn_w, btn_h = 140, 48
        self.btn_higher = pygame.Rect(width // 2 - btn_w - 20, height - 90, btn_w, btn_h)
        self.btn_lower = pygame.Rect(width // 2 + 20, height - 90, btn_w, btn_h)

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_medium = pygame.font.SysFont(None, 30)
        self.font_small = pygame.font.SysFont(None, 24)

        gap = 30
        self.left_x = width // 2 - self.CARD_W - gap
        self.right_x = width // 2 + gap
        self.card_y = 100
```

```python
# ---------------------------------------------------------------- logic
def streak_multiplier(self):
    """x1 on the first win, then x2, x3 ... capped at MAX_MULTIPLIER."""
    return min(max(self.streak, 1), self.MAX_MULTIPLIER)

def evaluate_guess(self, guess):
    """Draws next card, evaluates prediction, starts the reveal."""
    self.next_card = self.deck.draw()
    cur, nxt = self.current_card, self.next_card

    # Compare by numeric rank (2 < ... < 10 < J < Q < K < A), never by string.
    if nxt.numeric_rank == cur.numeric_rank:
        outcome = "PUSH"
    elif guess == "HIGHER":
        outcome = "CORRECT" if nxt.numeric_rank > cur.numeric_rank else "WRONG"
    else:
        outcome = "CORRECT" if nxt.numeric_rank < cur.numeric_rank else "WRONG"

    if outcome == "CORRECT":
        self.streak += 1
        self.multiplier = self.streak_multiplier()
        self.score += self.multiplier
        self.pending_msg = (
            f"CORRECT! {nxt.rank_str} vs {cur.rank_str}  "
            f"+{self.multiplier} (streak {self.streak}, x{self.multiplier})"
        )
        self.pending_color = (80, 220, 80)
    elif outcome == "WRONG":
        self.streak = 0
        self.multiplier = 1
        self.score = max(0, self.score - 1)
        self.pending_msg = f"WRONG! {nxt.rank_str} vs {cur.rank_str}  streak reset"
        self.pending_color = (235, 75, 75)
    else:  # PUSH: score and streak untouched
        self.pending_msg = f"PUSH! {nxt.rank_str} vs {cur.rank_str}  score and streak kept"
        self.pending_color = (240, 200, 70)

    self.state = self.REVEALING
    self.reveal_start = pygame.time.get_ticks()
    self.status_msg = "Revealing..."
    self.status_color = (220, 220, 220)
```

```
def handle_event(self, event):
    if self.state != self.WAITING:
        return
    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.btn_higher.collidepoint(event.pos):
            self.evaluate_guess("HIGHER")
        elif self.btn_lower.collidepoint(event.pos):
            self.evaluate_guess("LOWER")

def update(self):
    if self.state != self.REVEALING:
        return
    elapsed = pygame.time.get_ticks() - self.reveal_start
    if elapsed >= self.FLIP_MS:
        self.status_msg = self.pending_msg
        self.status_color = self.pending_color
    if elapsed >= self.REVEAL_MS:
        self.current_card = self.next_card
        self.next_card = None
        self.state = self.WAITING

# ------------------------------------------------------------ drawing
def _draw_flip(self, screen, elapsed):
    """Right-hand card: back shrinks to a sliver, then the face grows out."""
    t = min(elapsed / self.FLIP_MS, 1.0)
    surf = pygame.Surface((self.CARD_W, self.CARD_H), pygame.SRCALPHA)
    if t < 0.5:
        Card_back = self.next_card.render_back
        Card_back(surf, 0, 0, self.CARD_W, self.CARD_H)
        scale = 1 - t * 2
    else:
        self.next_card.render(surf, 0, 0, self.CARD_W, self.CARD_H)
        scale = (t - 0.5) * 2
    w = max(2, int(self.CARD_W * scale))
    scaled = pygame.transform.smoothscale(surf, (w, self.CARD_H))
    screen.blit(scaled, (self.right_x + (self.CARD_W - w) // 2, self.card_y))

def _button(self, screen, rect, fill, label):
    enabled = self.state == self.WAITING
    pygame.draw.rect(screen, fill if enabled else (90, 90, 90), rect, border_radius=8)
    pygame.draw.rect(screen, (220, 220, 220), rect, width=2, border_radius=8)
    surf = self.font_medium.render(label, True, (255, 255, 255) if enabled else (170, 170, 170))
    screen.blit(surf, (rect.centerx - surf.get_width() // 2, rect.centery - surf.get_height() // 2))
```

```
def render(self, screen):
    screen.fill((25, 80, 45))

    title_surf = self.font_title.render("High-Low Card Predictor", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 25))

    score_surf = self.font_medium.render(f"Score: {self.score}", True, (255, 220, 80))
    screen.blit(score_surf, (30, 30))
    streak_surf = self.font_small.render(
        f"Streak: {self.streak}  (x{self.streak_multiplier() if self.streak else 1})", True, (255, 200, 120)
    )
    screen.blit(streak_surf, (30, 58))

    rem_surf = self.font_small.render(f"Deck: {self.deck.remaining} left", True, (210, 210, 210))
    screen.blit(rem_surf, (self.width - rem_surf.get_width() - 30, 35))

    # Side-by-side cards with captions
    left_label = "PREVIOUS" if self.state == self.REVEALING else "CURRENT"
    right_label = "NEW CARD" if self.state == self.REVEALING else "NEXT"
    for label, x in ((left_label, self.left_x), (right_label, self.right_x)):
        cap = self.font_small.render(label, True, (200, 230, 200))
        screen.blit(cap, (x + self.CARD_W // 2 - cap.get_width() // 2, self.card_y - 24))

    self.current_card.render(screen, self.left_x, self.card_y, self.CARD_W, self.CARD_H)
    if self.state == self.REVEALING:
        self._draw_flip(screen, pygame.time.get_ticks() - self.reveal_start)
    else:
        self.current_card.render_back(screen, self.right_x, self.card_y, self.CARD_W, self.CARD_H)

    status_surf = self.font_small.render(self.status_msg, True, self.status_color)
    screen.blit(status_surf, (self.width // 2 - status_surf.get_width() // 2, 310))

    self._button(screen, self.btn_higher, (40, 140, 60), "HIGHER")
    self._button(screen, self.btn_lower, (170, 50, 50), "LOWER")
```

Main.py

```python
import pygame
from game.game_engine import GameEngine

WIDTH, HEIGHT = 650, 440
FPS = 60


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("High-Low Card Predictor - Pygame Edition")
    clock = pygame.time.Clock()

    engine = GameEngine(WIDTH, HEIGHT)

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.update()
        engine.render(screen)

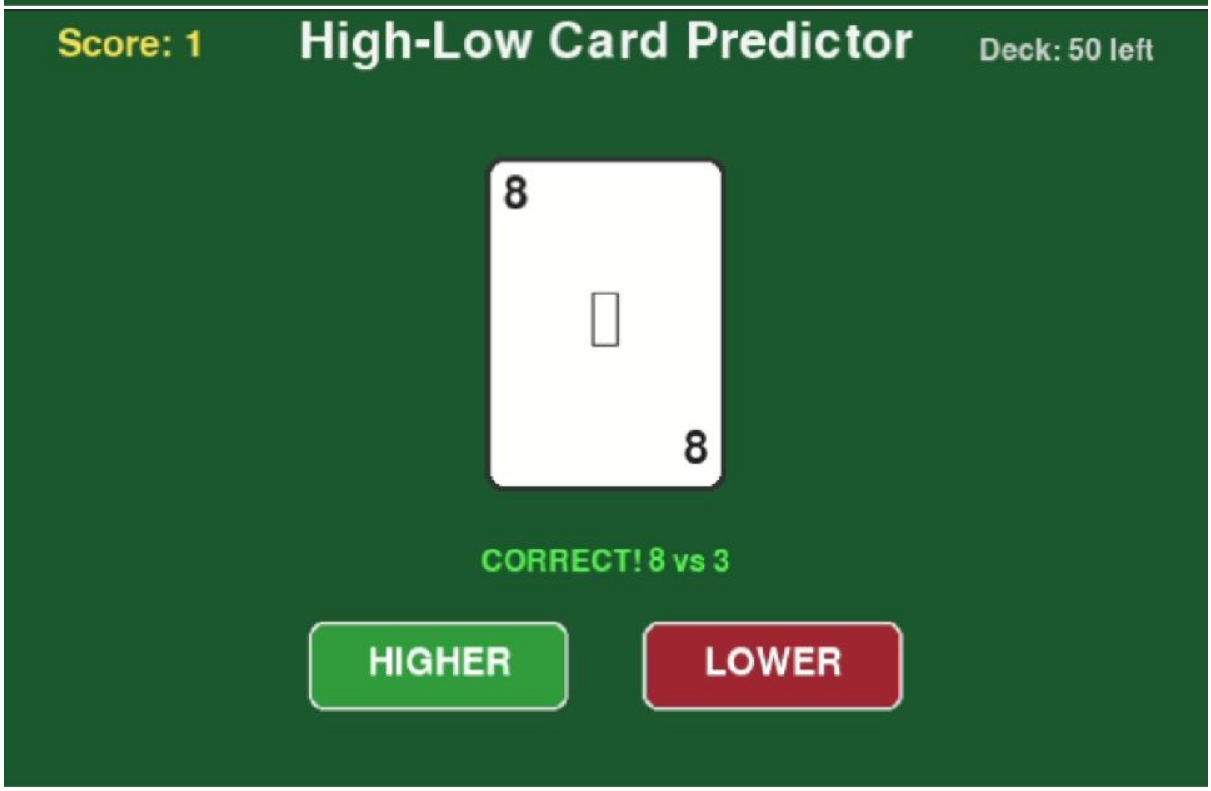
        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```

BEFORE


Score: 0
High-Low Card Predictor
Deck: 51 left
3
3
Will the next card be HIGHER or LOWER?
HIGHER
LOWER


Score: 1
High-Low Card Predictor
Deck: 50 left
8
8
CORRECT! 8 vs 3
HIGHER
LOWER

AFTER:





Chat link:
https://claude.ai/share/a90c84cd-1905-42c0-8089-350c63072d2f